Phiên bản: Zurich

NHÀ PHÁT TRIỂN ỨNG DỤNG SERVICENOW

Lập Trình Phía Client

BÀI VIẾT (1 TRÊN 21)

Mục Tiêu Lập Trình Phía Client

Trong module này bạn sẽ học:

- Mô tả mục đích của client-side script và đưa ví dụ về những gì client-side scripts có thể làm
- Tạo và kiểm thử Client Scripts
- Tạo và kiểm thử UI Policy scripts
- Sử dụng GlideForm và GlideUser APIs trong scripts
- Xác định nên dùng UI Policy scripts hay Client Scripts

BÀI VIẾT (2 TRÊN 21)

Về Module Học Tập Này

QUAN TRỌNG: Nội dung trong module học tập này được cập nhật lần cuối cho phiên bản San Diego ServiceNow và không được cập nhật cho phiên bản Utah. Bạn có thể thấy sự khác biệt giữa phiên bản Utah và nội dung trong module học tập này.

Nhiều ví dụ được sử dụng trên các trang khái niệm trong module học tập này. Bạn không cần cố gắng tái tạo các ví dụ. Bạn sẽ phát triển ứng dụng NeedIt trong các bài tập thực hành.

Bài tập được chỉ ra theo ba cách:

- Biểu tượng Bài tập trong khung Điều hướng.
- Biểu tượng Bài tập và chữ Exercise ở đầu trang.
- Chữ Exercise hoặc chữ Challenge trong tiêu đề trang.

Ứng dụng NeedIt cho phép người dùng yêu cầu dịch vụ từ nhiều phòng ban. Bạn sẽ sử dụng source control để bắt đầu với tất cả các file ứng dụng NeedIt cần thiết cho module học tập này.

BÀI TẬP (3 TRÊN 21)

Bài tập: Fork Repository và Import Ứng dụng cho Module Lập Trình Phía Client

ServiceNow sử dụng GitHub để cung cấp các repository ứng dụng để sao chép và sử dụng với nội dung học tập của Developer Site. Các repository chứa tags, là các tập hợp cố định của file ứng dụng, để bắt đầu với một ứng dụng đã xây dựng một phần. Bằng cách sao chép và import repository do ServiceNow cung cấp vào Personal Developer Instance (PDI) của bạn, bạn sẽ có tất cả các file cần thiết cho các bài tập thực hành.

GHI CHÚ: Xem Hướng dẫn GitHub (/dev.do#!/guide/utah/now-platform/github-guide/github-and-the-developer-site-training-guide-introduction) để biết thêm thông tin về cách ServiceNow sử dụng GitHub với nội dung học tập Developer Program và xem video về cách fork một repository và import ứng dụng.

Trong bài tập này, bạn sẽ:

1. Fork repository ServiceNow vào tài khoản GitHub của bạn.
2. Import ứng dụng vào PDI của bạn từ bản fork repository.

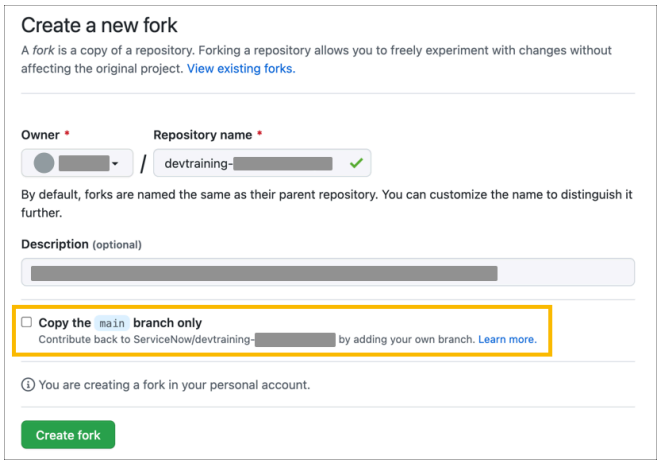
QUAN TRỌNG: Nếu bạn đã fork và import repository, bạn có thể tiến hành bài tập tiếp theo, nơi bạn sẽ tạo branch từ tag để tải các file ứng dụng vào PDI. Các file ứng dụng NeedIt cần thiết để hoàn thành module.

Fork Repository

Trong phần này của bài tập, bạn sẽ tạo một bản fork cá nhân của repository ứng dụng để sử dụng với nội dung học tập của Developer Site.

1. Trong trình duyệt web, mở github.com (https://github.com/).
2. Nếu bạn có tài khoản GitHub, đăng nhập. Nếu không, đăng ký tài khoản mới.
3. Sau khi đăng nhập, mở repository NeedIt (https://github.com/ServiceNow/devtraining-needit-utah).
4. Nhấp nút Fork (

) để tạo bản sao repository trong tài khoản GitHub của bạn.
5. Trên trang Create a new fork, bỏ chọn tùy chọn Copy the main branch only.



6. Chọn tài khoản GitHub cá nhân làm Owner cho fork, sau đó nhấp nút Create fork.

7. Xác minh URL cho bản fork repository tương tự: <YourGitHubUsername>/devtraining-application-release.

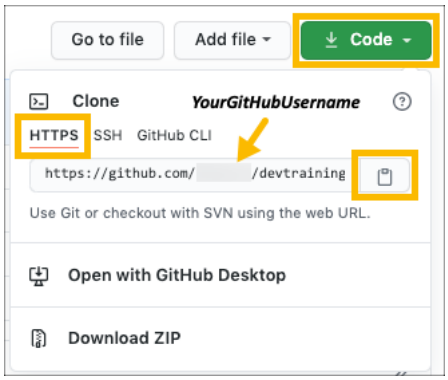
8. Sao chép URL của repository đã fork.

   1. Nhấp nút Code.

   2. Đảm bảo URL chứa tên GitHub của bạn, không phải ServiceNow.

   3. Đảm bảo HTTPS đã được chọn. Nếu không, chọn tab HTTPS trong flyout Clone.

   4. Nhấp nút Copy to clipboard ( ).



GHI CHÚ: Bạn sẽ sử dụng URL đã sao chép để cấu hình kết nối tới repository đã fork trong phần tiếp theo.

Import Ứng dụng từ Repository Đã Fork

Trong phần này của bài tập, bạn sẽ import repository ứng dụng vào ServiceNow. Như một phần của quy trình, trước tiên bạn sẽ tạo bản ghi Credential cho tài khoản GitHub, sau đó sử dụng Studio để import repository ứng dụng vào PDI của bạn.

1. Đăng nhập vào PDI với tư cách admin. Nếu bạn chưa có PDI, mở ServiceNow Developer Site (https://developer.servicenow.com) để lấy Utah PDI.

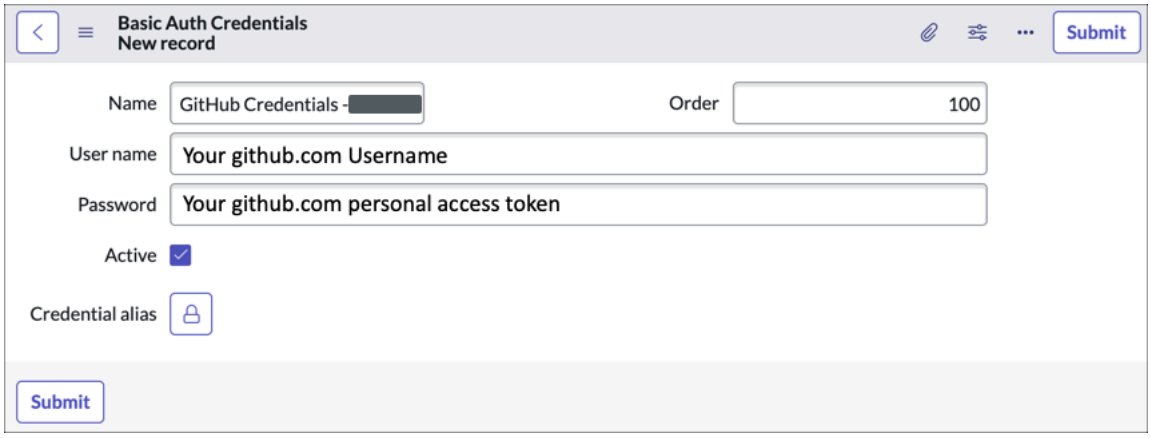
   GHI CHÚ: Xem Hướng dẫn Personal Developer Instance (PDI) (/dev.do#!/guide/utah/now-platform/pdi-guide/personal-developer-instance-guide-introduction) để biết hướng dẫn cách lấy PDI.

2. Tạo bản ghi Credential cho kết nối GitHub.

   QUAN TRỌNG: Bản ghi Credential chỉ cần tạo một lần. Nếu bạn đã tạo bản ghi credential trong bài tập khác, vui lòng bỏ qua bước này.

   1. Sử dụng menu All để mở Connections & Credentials > Credentials.
   2. Nhấp nút New.
   3. Trong danh sách What type of Credentials would you like to create?, nhấp liên kết Basic Auth Credentials.
   4. Cấu hình bản ghi Credential.

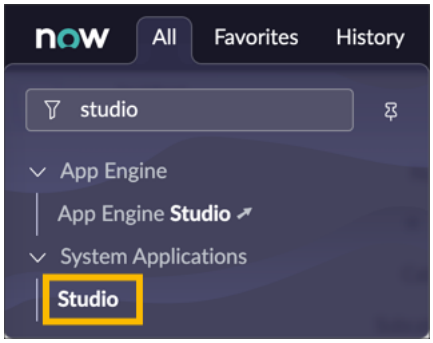
1. Name: GitHub Credentials - <Tên GitHub của bạn>
   User name: <Tên GitHub của bạn>
   Password: <Personal access token GitHub của bạn>



QUAN TRỌNG: GitHub yêu cầu personal access tokens để truy cập repositories từ các nền tảng khác, như ServiceNow. Personal access token được sử dụng thay cho mật khẩu khi xác thực. Xem phần Authenticating to GitHub (/dev.do#!/guides/utah/developer-program/github-guide/using-servicenow-provided-application-repositories#authenticating-to-github) trong Hướng dẫn GitHub để biết cách tạo GitHub personal access token.

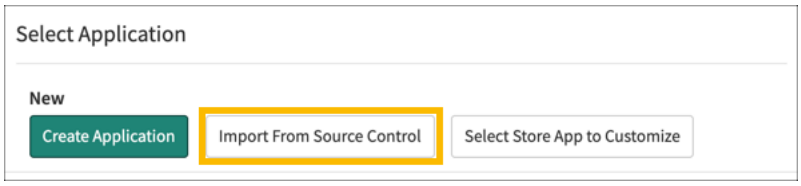
5. Nhấp nút Submit.

3. Sử dụng menu All để mở System Applications > Studio.



4. Studio mở trong tab trình duyệt mới.

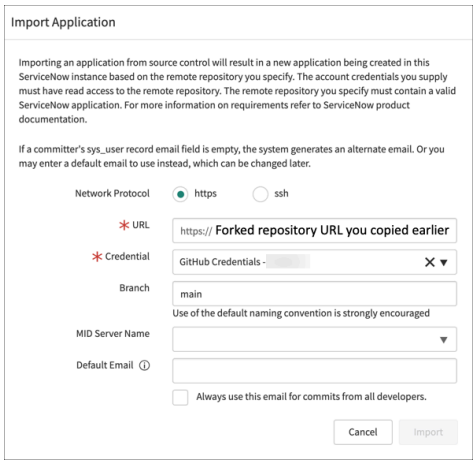
5. Trong hộp thoại Select Application, nhấp nút Import From Source Control.



6. Trong hộp thoại Import Application, cấu hình kết nối tới repository đã fork.

   1. URL: <URL bạn đã sao chép cho bản fork repository>
      Credential: GitHub Credentials - <Tên GitHub của bạn>
      Branch: main

GHI CHÚ: Khi bạn thay đổi giá trị Branch thành main, một thông báo cho biết việc sử dụng quy ước đặt tên mặc định được khuyến khích mạnh mẽ. Giá trị trong trường Branch phải tồn tại trong repository. Các repository đào tạo của Developer Site đều có branch main, nên được sử dụng thay cho giá trị mặc định.



7. Nhấp nút Import.

8. Khi import ứng dụng hoàn tất, nhấp nút Select Application.

GHI CHÚ: Nếu kết nối thất bại, có thể bạn đã nhập URL repository ServiceNow trong trường URL thay vì URL repository đã fork, hoặc bạn đã bật xác thực hai yếu tố trên tài khoản GitHub. Xem Troubleshooting GitHub Issues (/dev.do#!/guide/utah/now-platform/github-guide/troubleshooting-github-issues) để biết hướng dẫn cách

khắc phục sự cố kết nối.

9. Trong hộp thoại Select Application, nhấp ứng dụng để mở chỉnh sửa trong Studio.

QUAN TRỌNG: Bạn sẽ không thấy bất kỳ file ứng dụng nào trong Studio cho đến khi tạo thành công branch từ tag trong bài tập tiếp theo.

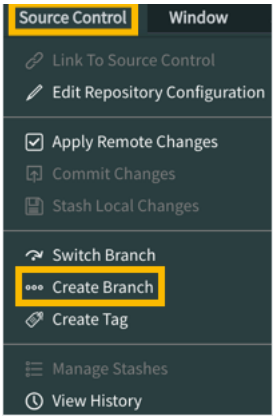
BÀI TẬP (4 TRÊN 21)

Bài tập: Tạo Branch cho Module Lập Trình Phía Client

Trong bài tập này, bạn sẽ tạo branch của ứng dụng NeedIt cho module Lập Trình Phía Client bao gồm các file ứng dụng được sử dụng trong module.

GHI CHÚ: Trước khi bắt đầu bài tập này, bạn cần đã fork và import repository NeedIt như mô tả trong Bài tập: Fork Repository và Import Ứng dụng cho Module Lập Trình Phía Client.

1. Nếu ứng dụng NeedIt chưa mở trong Studio từ bài tập trước, hãy mở ngay.
    1. Trong cửa sổ trình duyệt ServiceNow chính, sử dụng menu All để mở System Applications > Studio.
    2. Trong hộp thoại Select Application, nhấp ứng dụng NeedIt.
2. Trong Studio, mở menu Source Control và chọn mục Create Branch.



3. Cấu hình branch.

1. Branch Name: ClientScriptsModule
   Create from Tag: LoadForClientScriptsModule

4. Nhấp nút Create Branch.

5. Nhấp nút Close.

6. Để tải các file ứng dụng trong tag, quay lại tab trình duyệt ServiceNow chính (không phải Studio) và nhấp nút reload của trình duyệt để làm mới trang.

GHI CHÚ: Nếu tạo branch thất bại, có thể bạn đã nhập URL repository ServiceNow trong trường URL thay vì URL repository đã fork hoặc đã bật xác thực hai yếu tố trên tài khoản GitHub. Xem Troubleshooting GitHub Issues (/dev.do#!/guide/utah/now-platform/github-guide/troubleshooting-github-issues) trong Hướng dẫn GitHub để biết hướng dẫn cách khắc phục sự cố kết nối GitHub.

BÀI VIẾT (5 TRÊN 21)

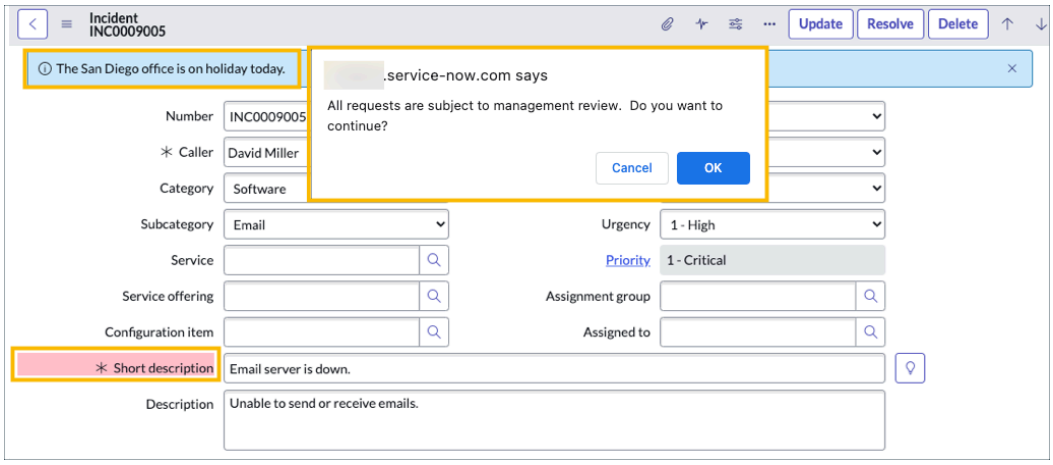
Giới Thiệu Lập Trình Phía Client

Scripts trong ServiceNow được chia thành hai loại:

- Phía Client
- Phía Server

Module học tập này về lập trình phía client. Client-side scripts thực thi trong trình duyệt của người dùng và được sử dụng để quản lý forms và các trường form. Ví dụ những gì client-side scripts có thể làm bao gồm:

- Đặt con trỏ vào trường form khi form được tải
- Tạo cảnh báo, xác nhận và thông báo
- Điền giá trị vào trường form phản hồi theo giá trị của trường khác
- Làm nổi bật trường form
- Xác thực dữ liệu form
- Sửa đổi các tùy chọn danh sách
- Ẩn/Hiện trường hoặc phần



Trong module này bạn sẽ học viết, kiểm thử và gỡ lỗi hại loại client-side scripts:

- Client Scripts
- UI Policy Scripts

BÀI VIẾT (6 TRÊN 21)

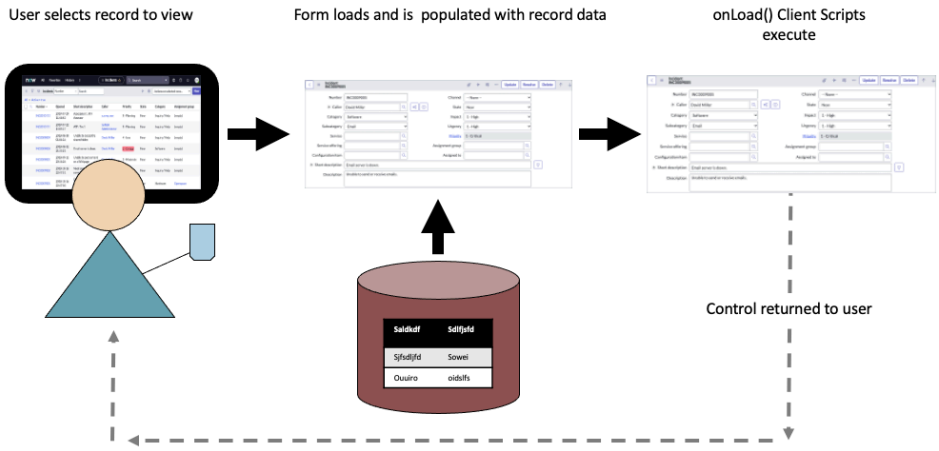
Các Loại Client Script

Client Script thực thi logic script phía client khi forms được:

- Tải (Loaded)
- Thay đổi (Changed)
- Gửi (Submitted)

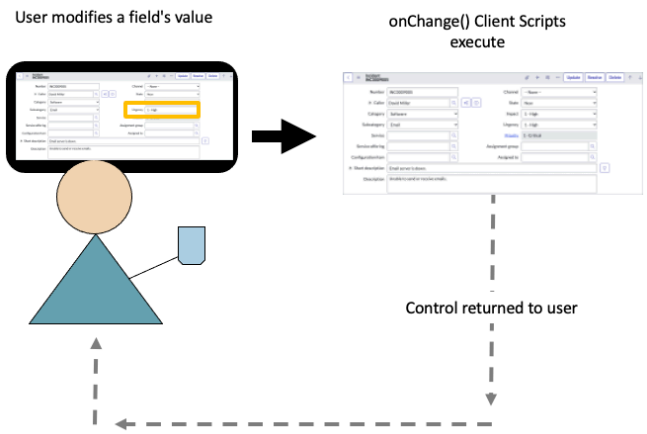
onLoad

onLoad Client Scripts thực thi logic script khi forms được tải. Sử dụng onLoad Client Scripts để thao tác giao diện hoặc nội dung form. Ví dụ, thiết lập thông báo ở mức trường hoặc form dựa trên sự hiện diện của giá trị. Sử dụng onLoad Client Scripts một cách tiết kiệm vì chúng ảnh hưởng đến thời gian tải form.



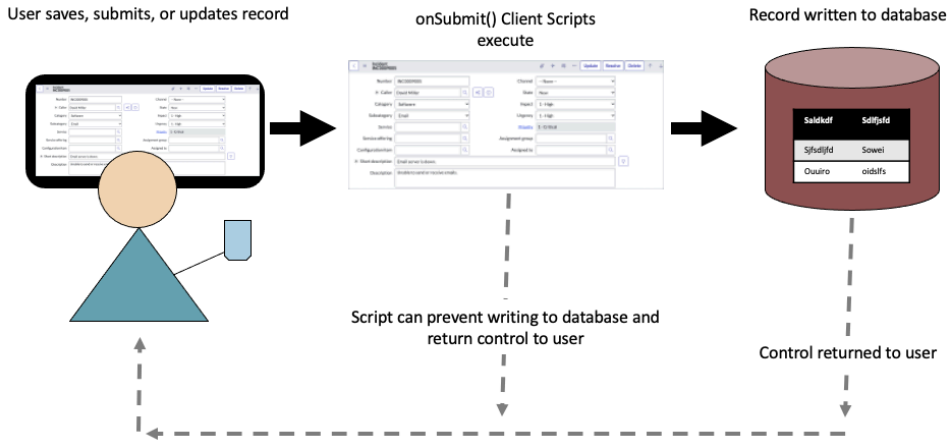
onChange

onChange Client Scripts thực thi logic script khi giá trị của một trường cụ thể thay đổi. Sử dụng onChange Client Scripts để phản hồi giá trị trường quan tâm và sửa đổi giá trị hoặc thuộc tính của trường khác. Ví dụ, nếu giá trị trường State thay đổi thành Closed Complete, tạo cảnh báo và đặt trường Description thành bắt buộc.



onSubmit

onSubmit Client Scripts thực thi logic script khi form được gửi. Sử dụng onSubmit Client Scripts để xác thực giá trị trường. Ví dụ, nếu người dùng gửi bản ghi Priority 1, script có thể tạo hộp thoại xác nhận thông báo cho người dùng rằng ban lãnh đạo được sao chép trên tất cả yêu cầu Priority 1.



GHI CHÚ: Trường Type có tùy chọn thứ tư: onCellEdit. Loại onCellEdit Client Script dành cho danh sách thay vì forms. onCellEdit Client Scripts không được đề cập trong module học tập này.

BÀI VIẾT (7 TRÊN 21)

Tạo Client Scripts

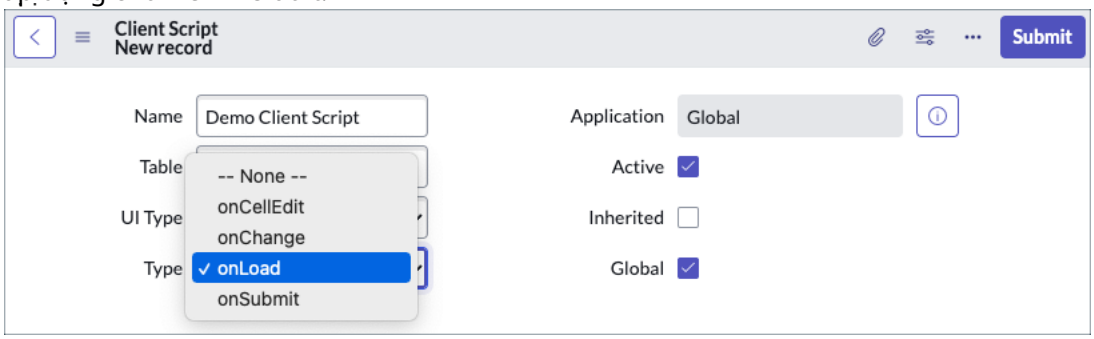
Quy trình thêm file vào ứng dụng trọng Studio không đổi bất kể loại file:

1. Nhấp liên kết Create Application File.
2. Chọn loại file mới, trọng trường hợp này là Client Script.
3. Cấu hình file mới.

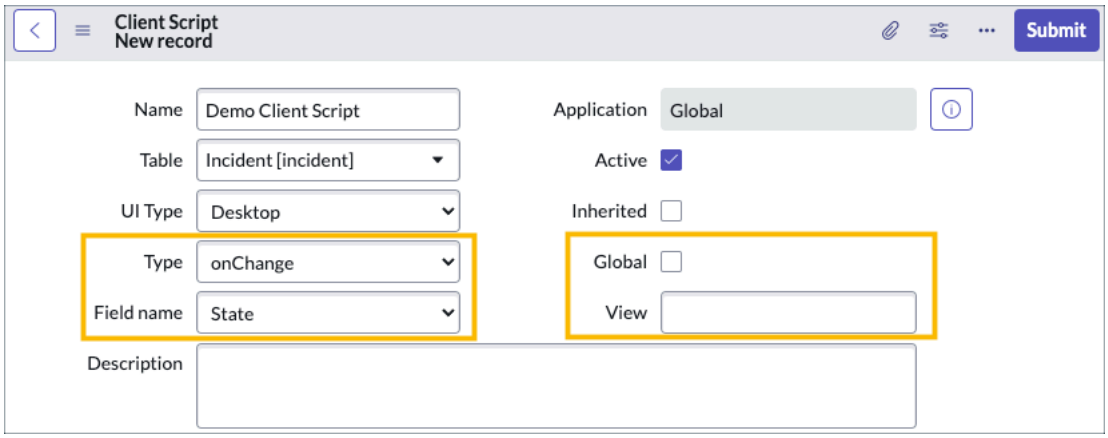
Cấu Hình Client Script

Như với bất kỳ script nào, cấu hình cho script biết khi nào thực thi. Các tùy chọn cấu hình Client Script là:

- Name: Tên của Client Script. Sử dụng quy ước đặt tên chuẩn để nhận diện custom scripts.
- Table: Bảng mà script áp dụng.
- UI Type: Chọn script thực thi cho Desktop và Tablet hoặc Mobile/Service Portal hoặc Tất cả.
- Type: Chọn khi nào script chạy: onChange, onLoad, hoặc onSubmit.
- Field Name: Chỉ dùng khi script phản hồi thay đổi giá trị trường (onChange); tên trường mà script áp dụng.
- Active: Kiểm soát script có được bật hay không. Scripts không active sẽ không thực thi.
- Inherited: Nếu được chọn, script áp dụng cho bảng được chỉ định và tất cả bảng kế thừa từ nó. Ví dụ, client script trên bảng Task cũng sẽ áp dụng cho Change, Incident, Problem và tất cả các bảng khác mở rộng từ Task.
- Global: Nếu Global được chọn, script áp dụng cho tất cả Views. Nếu trường Global không được chọn, bạn phải chỉ định View.
- View: Chỉ định View mà script áp dụng. Trường View hiển thị khi Global không được chọn. Script chỉ có thể tác động lên các trường thuộc View form đã chọn. Nếu trường View để trống, script áp dụng cho view Default.



Trường Field name có sẵn cho onChange Client Scripts. Trường View có sẵn khi tùy chọn Global không được chọn.



MẸO CHO DEVELOPER: Khi logic phía client được sửa đổi, hãy tải lại cửa sổ trình duyệt ServiceNow chính để đảm bảo logic mới nhất được tải lên.

BÀI VIẾT (8 TRÊN 21)

Trường Script

Khi giá trị type được thiết lập, một template script tự động được chèn vào trường Script.

Template Script onLoad

Template script onLoad:

Hàm onLoad không có tham số được truyền vào. Như chỉ dẫn trong comment, thêm logic script của bạn trong hàm onLoad. Đó là thực hành tốt nhất để ghi chú code, vì vậy hãy thêm comments để giải thích script làm gì. Bạn trong tương lai sẽ cảm ơn bạn vì script được ghi chú rõ ràng.

Script ví dụ này tạo cảnh báo khi người dùng yêu cầu tải form cho một bản ghi. Người dùng không thể tương tác với form cho đến khi onLoad Client Scripts hoàn tất thực thi.

Script

```
function onLoad() {
    //Type appropriate comment here, and begin script below
    alert("Thank you for loading the form.  When you close this alert you will be able to interact with the form.");
}
```

Template Script onSubmit

Template script onSubmit:

Script

```
function onSubmit() {
   //Type appropriate comment here, and begin script below

}
```

Hàm onSubmit không có tham số được truyền vào. Như chỉ dẫn trong comment, thêm logic script của bạn trong hàm onSubmit.

Script ví dụ này tạo cảnh báo khi người dùng lưu bản ghi NeedIt. Bản ghi không được gửi đến server cho đến khi onSubmit Client Scripts hoàn tất thực thi và trả về true.

Script

```
function onSubmit() {
   //Generate an alert to thank the user for submitting a record
    alert("Thank you for submitting a record.");
}
```

Template Script onChange

Template Script onChange:

Script

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
   if (isLoading || newValue === '') {
      return;
   }

   //Type appropriate comment here, and begin script below

}
```

Hàm onChange được ServiceNow tự động truyền năm tham số. Mặc dù bạn không cần làm gì để truyền các tham số, bạn có thể sử dụng chúng trong script của bạn.

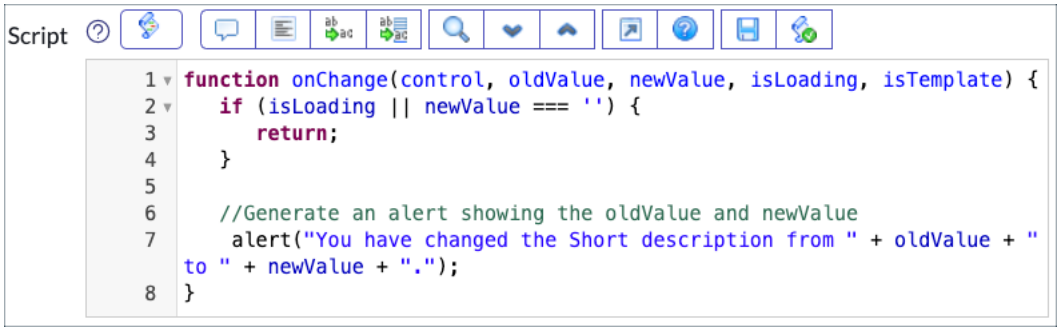
- control: trường mà Client Script được cấu hình.
- oldValue: giá trị của trường khi form được tải và trước khi thay đổi.
- newValue: giá trị của trường sau khi thay đổi.
- isLoading: giá trị boolean cho biết thay đổi có đang xảy ra như một phần của việc tải form không. Giá trị là true nếu thay đổi do tải form. Khi forms tải, tất cả giá trị trường trên form thay đổi khi bản ghi được tải vào form.
- isTemplate: giá trị boolean cho biết thay đổi có xảy ra do điền giá trị trường bởi template hay không. Giá trị là true nếu thay đổi do điền từ template.

Khi người dùng chọn bản ghi để tải, form và layout form được render trước, sau đó các trường được điền giá trị từ cơ sở dữ liệu. Từ góc nhìn kỹ thuật, tất cả giá trị trường bị thay đổi, từ không có gì sang giá trị của bản ghi, khi form tải. Câu lệnh if trong template giả định rằng onChange Client scripts không nên thực thi logic script khi form tải. onChange Client Script cũng dừng thực thi nếu newValue của trường không có giá trị. Tùy vào trường hợp sử dụng, bạn có thể sửa đổi hoặc xóa câu lệnh if. Ví dụ:

```
//Dừng thực thi script nếu thay đổi giá trị trường do Template gây ra
        if(isLoading || newValue === '' || isTemplate) {
           return;
        }
```

Trong ví dụ này, onChange Client Script thực thi vì thay đổi của trường Short description giá trị:

Khi người dùng thay đổi giá trị trường Short description trên form, logic script onChange thực thi. Ví dụ tạo cảnh báo cho biết giá trị trường Short description đã thay đổi từ giá trị trường có khi form được tải sang giá trị mới trên form.



Giá trị trường Short description chỉ thay đổi trên form. Các thay đổi không được gửi đến cơ sở dữ liệu cho đến khi người dùng lưu, cập nhật hoặc gửi bản ghi.

Điều quan trọng cần biết là giá trị của oldValue được thiết lập khi form tải. Bất kể trường Short description thay đổi bao nhiêu lần, oldValue vẫn giữ nguyên cho đến khi form được tải lại từ cơ sở dữ liệu.

1. Form tải:
    1. oldValue = hello
    2. newValue không có giá trị
2. Người dùng thay đổi giá trị trong trường Short description thành bye:
    1. oldValue = hello
    2. newValue = bye
3. Người dùng thay đổi giá trị trong trường Short description thành greetings:
    1. oldValue = hello
    2. newValue = greetings
4. Người dùng lưu form và tải lại trang form:
    1. oldValue = greetings
    2. newValue không có giá trị

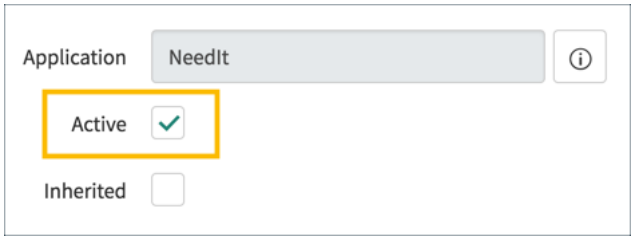
BÀI TẬP (9 TRÊN 21)

Bài tập: Kiểm Thử Các Loại Client Script

Trong bài tập này bạn sẽ kiểm thử các loại Client Script onLoad, onChange và onSubmit sử dụng Client Scripts từ repository GitHub bạn đã fork.

Chuẩn Bị

1. Nếu ứng dụng NeedIt chưa mở trong Studio từ bài tập trước, hãy mở ngay.
    1. Trong cửa sổ trình duyệt ServiceNow chính, mở menu All.
    2. Trong trường Filter, nhập Studio.
    3. Mở System Applications > Studio.
    4. Trong hộp thoại Select Application, nhấp ứng dụng NeedIt.
2. Kích hoạt NeedIt onLoad Example Client Script.
    1. Trong Studio, dùng Application Explorer để mở Client Development > Client Scripts > NeedIt onLoad Example.
    2. Chọn tùy chọn Active.

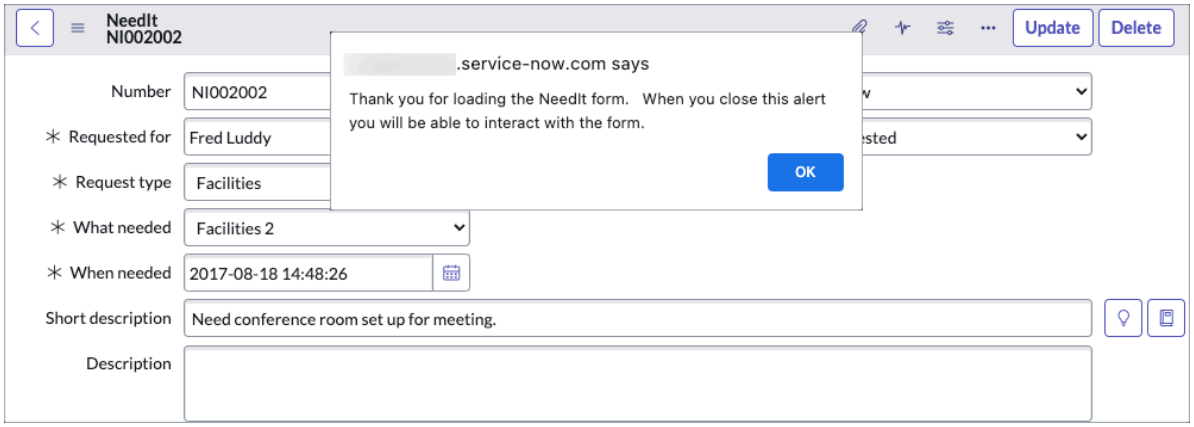


    3. Kiểm tra cấu hình để xem script dùng cho bảng nào.
3. Kiểm tra trường Script để xác định script làm gì.
4. Nhấp nút Update.

5. Sử dụng cùng chiến lược, kích hoạt NeedIt onSubmit Example và NeedIt onChange Example Client Scripts. Với onChange Client Script, ghi chú giá trị trong trường Field name.

Kiểm Thử Client Scripts

1. Trong cửa sổ trình duyệt ServiceNow chính (không phải Studio), dùng menu All để mở NeedIt > All.
2. Kiểm thử NeedIt onLoad Example Client Script.
   1. Mở bản ghi NeedIt bạn chọn để chỉnh sửa.
   2. Cảnh báo có xuất hiện không? Nếu không, đảm bảo bạn đã kích hoạt NeedIt onLoad Example Client Script và lưu thay đổi.



   3. Với cảnh báo vẫn mở, thử sửa đổi giá trị trường trên form. Bạn không thể sửa đổi form cho đến khi script NeedIt onLoad Example hoàn tất thực thi. onLoad Client scripts với cảnh báo ngăn quyền kiểm soát form được trao cho người dùng cho đến khi cảnh báo được đóng.
   4. Nhấp nút OK để đóng cảnh báo.
3. Kiểm thử NeedIt onChange Example Client Script.
   1. Chỉnh sửa bất kỳ trường nào trên form trừ trường Short description.
   2. Không có cảnh báo nào xuất hiện.
   3. Chỉnh sửa giá trị trong trường Short description rồi nhấp vào form bên ngoài trường.
   4. Cảnh báo sẽ xuất hiện hiển thị oldValue và newValue. Nếu không có cảnh báo, đảm bảo bạn đã kích hoạt NeedIt onChange Example Client Script và lưu thay đổi.
   5. Nhấp nút OK để đóng cảnh báo.
   6. Chỉnh sửa trường Short description lần nữa rồi nhấp bên ngoài trường.
   7. Trong cảnh báo, lưu ý oldValue không thay đổi.
   8. Nhấp nút OK để đóng cảnh báo.
4. Kiểm thử NeedIt onSubmit Example Client Script.
   1. Nhấp nút Update trên header form.
   2. Cảnh báo có xuất hiện không? Nếu không, đảm bảo bạn đã kích hoạt NeedIt onSubmit Example Client Script và lưu thay đổi.
   3. Nhấp nút OK để đóng cảnh báo.
5. Quay lại Studio và vô hiệu hóa Client Scripts:

- NeedIt onLoad Example
- NeedIt onChange Example

- NeedIt onSubmit Example

BÀI VIẾT (10 TRÊN 21)

Lớp GlideForm (g_form)

GlideForm (https://developer.servicenow.com/dev.do#!/reference/api/utah/client/c_GlideFormAPI) API phía client cung cấp các phương thức để quản lý form và trường form bao gồm các phương thức để:

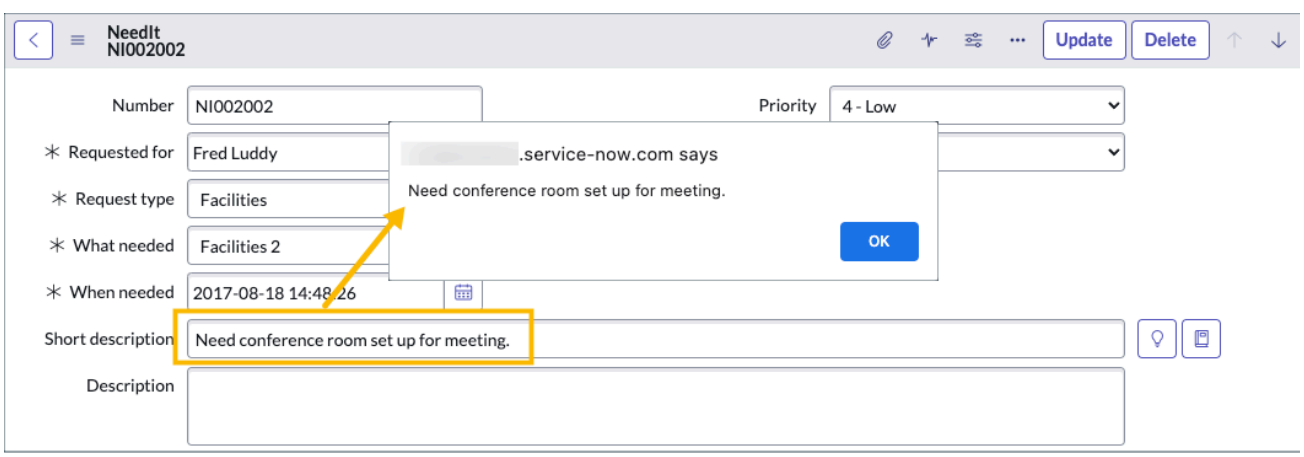
- Lấy giá trị trường trên form
- Ẩn trường
- Đặt trường chỉ đọc
- Viết thông báo trên form hoặc trường
- Thêm trường vào danh sách lựa chọn
- Xóa trường khỏi danh sách lựa chọn

Các phương thức GlideForm được truy cập qua đối tượng toàn cục g_form chỉ có sẵn trong scripts phía client. Để sử dụng các phương thức từ lớp GlideForm, dùng cú pháp:

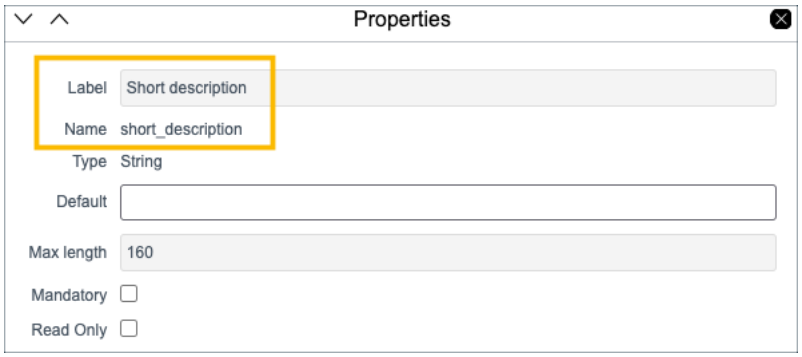
```
g_form.<method name>
```

Ví dụ, GlideForm API có phương thức gọi là getValue(). Phương thức getValue lấy giá trị của trường trên form (không phải giá trị trong cơ sở dữ liệu). Script ví dụ lấy giá trị trường Short description từ form và hiển thị giá trị trường Short description trong cảnh báo.

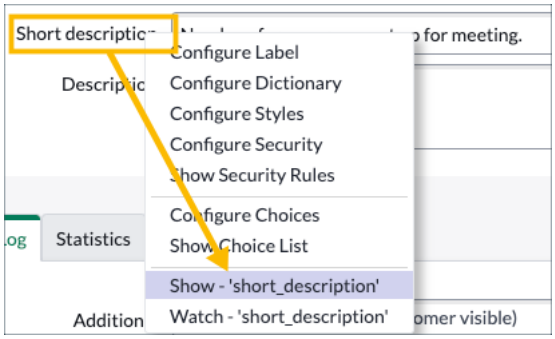
```
alert(g_form.getValue('short_description'));
```



Lưu ý trên form, tên trường là Short description nhưng script tham chiếu cùng trường là short_description. Tại sao hai giá trị này khác nhau? Tất cả trường bảng đều có Label và Name. Label là phiên bản thân thiện với người dùng của tên trường hiển thị trên forms. Name là giá trị dùng để tham chiếu trường trong script. Names luôn viết thường và không chứa khoảng trắng.



Có nhiều cách để tìm giá trị Name và Label cho trường như xem định nghĩa bảng, xem thuộc tính trong Form Designer, hoặc xem trong Dictionary. Cách nhanh nhất để tìm giá trị Name và Label khi form đang mở là nhấp chuột phải vào Label của trường trên form. Label xuất hiện trên form và Name xuất hiện trong menu nhấp chuột phải.



Ngoài phương thức getValue(), các phương thức GlideForm thường dùng khác bao gồm:

- *addOption()*
- *clearOptions()*
- *addInfoMessage()*
- *addErrorMessage()*
- *showFieldMsg()*
- *clearMessages()*
- *getSections()*
- *getSectionName()*

Để xem danh sách đầy đủ API phía client, truy cập Developer Site API Reference (https://developer.servicenow.com/dev.do#!/reference/api/utah/client/).

BÀI VIẾT (11 TRÊN 21)

Lớp GlideUser (g_user)

GlideUser (https://developer.servicenow.com/dev.do#!/reference/api/utah/client/c_GlideUserAPI) API cung cấp các phương thức và thuộc tính để tìm thông tin về người dùng đang đăng nhập và vai trò của họ. Các trường hợp sử dụng điển hình là cá nhân hóa phản hồi cho người dùng và kiểm tra vai trò. Lưu ý rằng xác thực phía client trọng bất kỳ ứng dụng web nào đều dễ bị bỏ qua.

GlideUser API có các thuộc tính và phương thức để:

- Lấy thông tin người dùng:
  - Tên
  - Họ tên đầy đủ
  - Họ
  - User ID
  - Tên đăng nhập
- Xác định người dùng có được gán vai trò cụ thể hay không

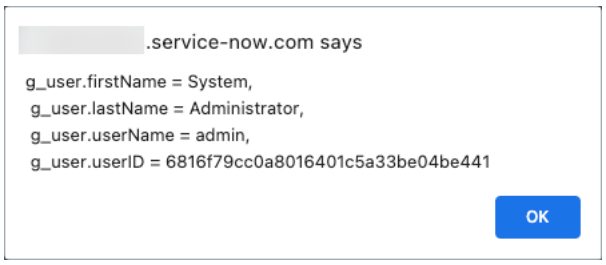
Các phương thức và thuộc tính GlideUser được truy cập qua đối tượng toàn cục g_user chỉ có sẵn trong scripts phía client. Để sử dụng phương thức và thuộc tính từ lớp GlideUser, dùng cú pháp:

```
g_user.<method or property name>
```

Ví dụ, GlideUser API có thuộc tính gọi là userName. Giá trị thuộc tính userName là tên đăng nhập của người dùng hiện tại. Script ví dụ cho thấy sự khác biệt giữa các thuộc tính firstName, lastName, userName và userID.

```
alert("g_user.firstName = " + g_user.firstName
          + ", \n g_user.lastName = " + g_user.lastName
          + ", \n g_user.userName = " + g_user.userName
          + ", \n g_user.userID = " + g_user.userID);
```

Cảnh báo được tạo bởi script là:



Thuộc tính g_user.userID chứa sys_id của bản ghi. Mỗi bản ghi có sys_id duy nhất 32 ký tự.

Mặc dù bạn có thể nối đầu ra g_user.firstName với g_user.lastName, phương thức tiện lợi g_user.getFullName() nối hai giá trị đó.

GlideUser API cũng có phương thức để xác định người dùng có vai trò cụ thể không. Ví dụ:

```
g_user.hasRole('client_script_admin');
```

Script ví dụ kiểm tra xem người dùng đang đăng nhập có khả năng tạo và chỉnh sửa Client Scripts (vai trò client_script_admin) hay không. Lưu ý script trả về true không chỉ khi người dùng hiện tại được gán vai trò mà còn khi người dùng có vai trò admin. Người dùng admin có tất cả vai trò được gán ngầm. Để kiểm tra người dùng hiện tại có vai trò được gán tường minh hay không, dùng phương thức hasRoleExactly():

```
g_user.hasRoleExactly('client_script_admin');
```

Để xem danh sách đầy đủ API phía client, truy cập Developer Site API Reference (https://developer.service-now.com/dev.do#!/reference/api/utah/client/).

BÀI TẬP (12 TRÊN 21)

Bài tập: Tạo Client Scripts

Trong bài tập này bạn sẽ viết và kiểm thử hai Client Scripts:

- Script để thiết lập giá trị danh sách lựa chọn What needed dựa trên giá trị trong trường Request type
- Đặt giá trị Requested for thành người dùng đang đăng nhập

Tạo NeedIt Request Type Options Client Script

1. Kiểm tra các trường Request type và What needed trên form NeedIt.
    1. Trong cửa sổ trình duyệt ServiceNow chính (không phải Studio), dùng menu All để mở NeedIt > All.
    2. Mở bản ghi NeedIt bạn chọn để chỉnh sửa.
    3. Nhấp trường Request type để xem các lựa chọn. Bạn sẽ thấy: Facilities, Human Resources và Legal.
    4. Nhấp trường What needed để xem các lựa chọn. Lưu ý các lựa chọn bao gồm phòng ban khác ngoài phòng ban được chọn trong trường Request type.
    5. Dùng menu All để mở NeedIt > Create New.
    6. Lưu ý giá trị mặc định cho trường Request type.
2. Quay lại Studio và tạo Client Script.
    1. Trong Studio, nhấp liên kết Create Application File.
    2. Trong trường Filter... nhập text Client HOẶC chọn Client Development từ danh mục trong khung bên trái.
    3. Chọn Client Script ở khung giữa làm loại file, sau đó nhấp nút Create.

GHI CHÚ: Nếu bạn bật annotations, bạn sẽ thấy annotation:

Client-scripts mới chạy ở chế độ strict, với truy cập DOM trực tiếp bị vô hiệu hóa. Truy cập jQuery, prototype và đối tượng window cũng bị vô hiệu hóa. Để vô hiệu hóa trên cơ sở từng script, cấu hình form và thêm trường "Isolate script". Để vô hiệu hóa tính năng này cho tất cả client-side scripts phạm vi toàn cục mới, thiết lập thuộc tính hệ thống "glide.script.block.client.globals" thành false.

Client Scripts sử dụng trong module này không truy cập các đối tượng mô tả trong annotation. Bạn có thể bỏ qua thông báo.

3. Cấu hình Client Script:

   1. Name: NeedIt Request Type Options
      Table: NeedIt [x_<mã_công_ty_của_bạn>_needit_needit]
      UI Type: All
      Type: onChange
      Field name: Request type
      Description: Chỉ hiển thị lựa chọn What needed khớp với giá trị Request type.

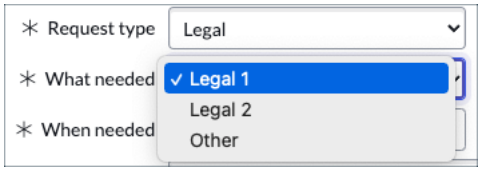
4. Thay thế nội dung trường Script bằng script này:

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
                    if ( newValue == '') {
                              return;
                    }

                    var whatneeded = g_form.getValue('u_what_needed');

                    // Clear all of the choices from the What needed field choice list
                    g_form.clearOptions('u_what_needed');

                    // If the value of the Request type field is hr, add
                    // two hr choices and other to the What needed field choice list
                    if(newValue == 'hr'){
                              g_form.addOption('u_what_needed','hr1','Human Resources 1');
                              g_form.addOption('u_what_needed','hr2','Human Resources 2');
                              g_form.addOption('u_what_needed','other','Other');
                    }
                    // If the value of the Request type field is facilities, add
                    // two facilities choices and other to the What needed field
                    // choice list
                    if(newValue == 'facilities'){
                              g_form.addOption('u_what_needed','facilities1','Facilities 1');
                              g_form.addOption('u_what_needed','facilities2','Facilities 2');
                              g_form.addOption('u_what_needed','other','Other');
                    }
                    // If the value of the Request type field is legal, add
                    // two legal choices and other to the What needed field
                    // choice list
                    if(newValue == 'legal'){
                              g_form.addOption('u_what_needed','legal1','Legal 1');
                              g_form.addOption('u_what_needed','legal2','Legal 2');
                              g_form.addOption('u_what_needed','other','Other');
                    }

                    // If the form is loading and it is not a new record, set the u_what_needed value to the
                    // value from the record before it was loaded
                    if(isLoading && !g_form.isNewRecord()){
                              g_form.setValue('u_what_needed', whatneeded);
                    }
          }
```

5. Nhấp nút Submit.

Kiểm thử NeedIt Request Type Options Client Script

1. Trong cửa sổ trình duyệt ServiceNow chính (không phải Studio), dùng menu All để mở NeedIt > All.
2. Mở bản ghi bạn chọn để chỉnh sửa. Nếu bạn đã mở bản ghi NeedIt, tải lại form để các thay đổi có hiệu lực.
3. Lưu ý giá trị trong trường Request type.
4. Nhấp vào trường What needed để mở danh sách lựa chọn. Các lựa chọn phải khớp với giá trị trong trường Request type cộng thêm Other. Ví dụ:



5. Chọn lựa chọn khác trong trường Request type và xác nhận lựa chọn What needed cũng thay đổi.

Tạo NeedIt Set Requested for Client Script

1. Quay lại Studio và tạo Client Script.
    1. Trong Studio, nhấp liên kết Create Application File.
    2. Trong trường Filter... nhập text Client HOẶC chọn Client Development từ danh mục trong khung bên trái.
    3. Chọn Client Script ở khung giữa làm loại file, sau đó nhấp nút Create.
2. Cấu hình Client Script:
    1. Name: NeedIt Set Requested for
       Table: NeedIt [x_<mã_công_ty_của_bạn>_needit_needit]
       UI Type: All
       Type: onLoad
       Description: Đặt Requested for thành người dùng đang đăng nhập cho bản ghi mới. Người dùng có thể thay đổi giá trị trường
3. Thay thế nội dung trường Script bằng script này:

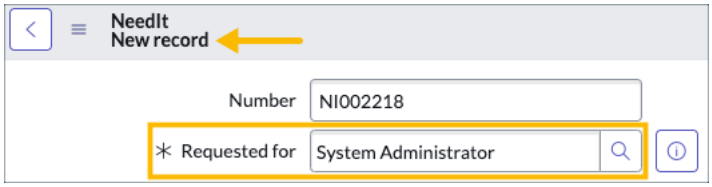
```
function onLoad() {
  //Check to see if the form is for a new record.  If it is a new record,
  //set the Requested for value to the currently logged in user.

    if(g_form.isNewRecord()){
        g_form.setValue('u_requested_for',g_user.userID);
    }
}
```

4. Nhấp nút Submit.

Kiểm thử NeedIt Set Requested for Client Script

1. Trong cửa sổ trình duyệt ServiceNow chính (không phải Studio), dùng menu All để mở NeedIt > Create New.

2. Kiểm tra trường Requested for. Giá trị sẽ tự động được đặt thành System Administrator.



3. Thay đổi giá trị Requested for thành Beth Anglin.

4. Thêm giá trị bạn chọn vào các trường bắt buộc (được chỉ bằng dấu * trên form).

5. Nhấp nút Submit.

6. Từ danh sách bản ghi, mở bản ghi bạn vừa tạo.

7. Kiểm tra giá trị trường Requested for.

BÀI VIẾT (13 TRÊN 21)

UI Policies

Giống Client Scripts, UI Policies là logic phía client quản lý hành vi form và trường form. Không giống Client Scripts, UI Policies không phải lúc nào cũng yêu cầu viết script.

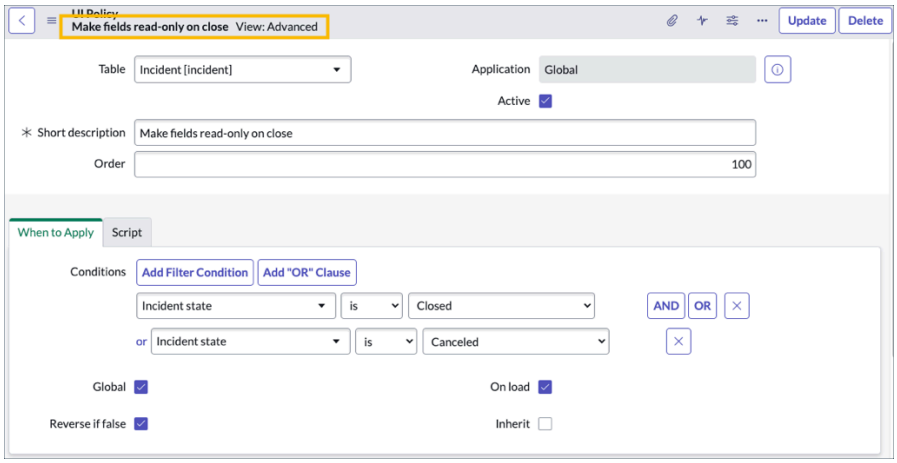
Tạo UI Policies

Quy trình thêm file vào ứng dụng trong Studio không đổi bất kể loại file:

1. Nhấp liên kết Create Application File.
2. Chọn loại file, trong trường hợp này là UI Policy.
3. Cấu hình file mới.

Cấu Hình UI Policy

UI Policies có hai views: Default và Advanced. Các trường trong cấu hình UI Policy khác nhau tùy thuộc vào View được chọn. View Advanced hiển thị tất cả các trường cấu hình. View Default hiển thị một tập con các trường.



- Table: Form (bảng) mà UI Policy áp dụng.
- Application: Xác định phạm vi của UI Policy.
- Active: Kiểm soát UI Policy có được bật hay không.
- Short description: Giải thích ngắn gọn UI Policy làm gì.
- Order: Nếu nhiều UI Policies tồn tại cho cùng bảng, dùng trường Order để đặt thứ tự đánh giá của các điều kiện UI Policy.
- Condition: Điều kiện phải đáp ứng để kích hoạt logic UI Policy.
- Global: Nếu Global được chọn, script áp dụng cho tất cả views của bảng. Nếu trường Global không được chọn, bạn phải chỉ định view.
- View: Chỉ định view mà script áp dụng. Trường View chỉ hiển thị khi Global không được chọn. Script chỉ có thể tác động lên trường thuộc view form đã chọn. Nếu trường View để trống, script áp dụng cho view Default.
- On load: Khi được chọn, trường điều kiện UI Policy được đánh giá khi form tải ngoài khi giá trị trường thay đổi. Khi không được chọn, Điều kiện UI Policy chỉ được đánh giá khi giá trị trường thay đổi.
- Reverse if false: Thực hiện hành động ngược lại khi trường Condition đánh giá false.
- Inherit: Khi được chọn, thực thi script cho forms có bảng mở rộng từ bảng của UI Policy.

MẸO CHO DEVELOPER: Nhập giá trị mô tả trong trường Short description vì UI Policies không có trường Name. Khi gỡ lỗi, nhận diện UI Policies bằng giá trị trường Short description.

Nếu trường Condition không có giá trị, điều kiện trả về true và logic UI Policy sẽ thực thi mỗi lần có thay đổi giá trị trường trên form.

Trường Order đặt thứ tự đánh giá điều kiện UI Policy cho các UI Policies cùng bảng. Thứ tự đánh giá từ số thấp nhất đến cao nhất (thứ tự tăng dần). Theo quy ước, giá trị trường Order là số tròn hàng trăm: 100, 200, 300 v.v. Điều này không bắt buộc.

MẸO CHO DEVELOPER: Tránh đặt thứ tự UI Policies là 1, 2, 3, v.v. Để khoảng cách giữa giá trị trường Order để có thể chèn UI Policy mới vào danh sách hiện có mà không cần sắp xếp lại UI Policies hiện có.

BÀI VIẾT (14 TRÊN 21)

UI Policy Actions

UI Policy Actions là logic phía client trong UI Policy dùng để thiết lập ba thuộc tính trường:

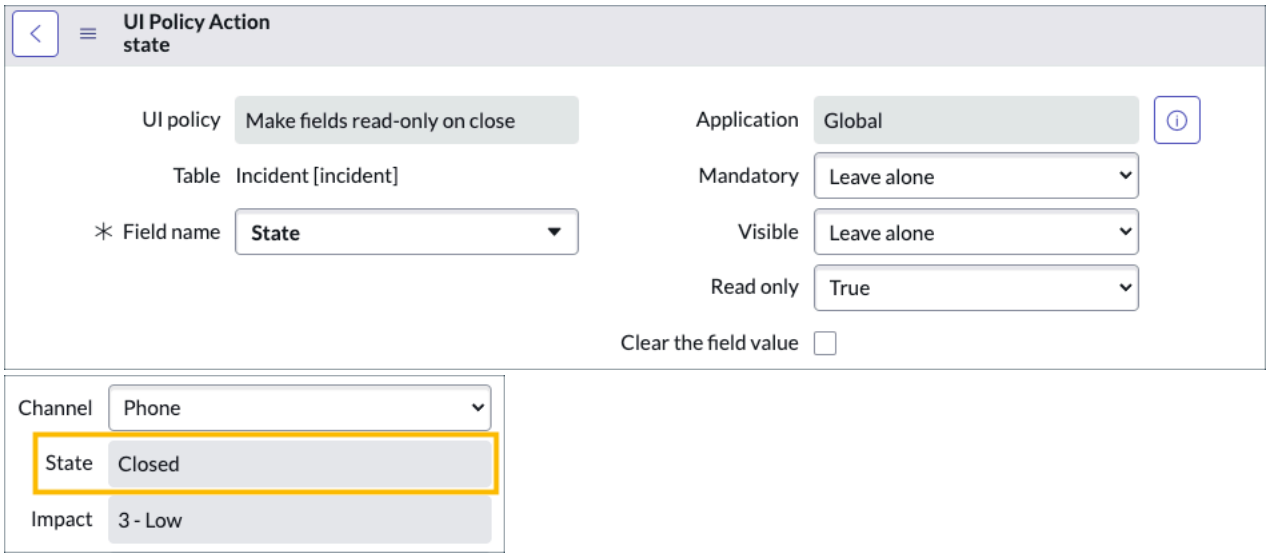
- Bắt buộc (Mandatory)
- Hiển thị (Visible)
- Chỉ đọc (Read only)

Mặc dù bạn có thể dùng scripts để thiết lập thuộc tính này sử dụng GlideForm (g_form) API, UI Policy Actions KHÔNG yêu cầu viết script để thiết lập thuộc tính trường.

Tạo UI Policy Actions

1. Trong Studio, tạo UI Policy hoặc mở UI Policy hiện có để chỉnh sửa.
2. Cuộn xuống danh sách liên quan UI Policy Actions. Nếu tạo UI Policy mới, UI Policy phải được lưu trước khi danh sách liên quan UI Policy Actions hiển thị.
3. Nhấp nút New.
4. Cấu hình UI Policy Action.
    1. Chọn Field name.
    2. Thiết lập giá trị trường Mandatory, Visible hoặc Read-only.
    3. True: Áp dụng thuộc tính cho trường.
       False: Không áp dụng thuộc tính cho trường.
       Leave alone: Thuộc tính không áp dụng cho trường.
5. Để xóa giá trị hiện có khỏi trường, chọn tùy chọn Clear the field value.
6. Nhấp nút Submit.

Khi điều kiện UI Policy kiểm tra true, UI Policy Actions được áp dụng. Trong ví dụ, trường State là Read only. Giá trị thuộc tính Mandatory và Visible không bị thay đổi bởi UI Policy Action.



Điều gì xảy ra khi điều kiện UI Policy kiểm tra false? Có hai kết quả có thể:

- Không thực hiện hành động
- Thực hiện hành động ngược lại

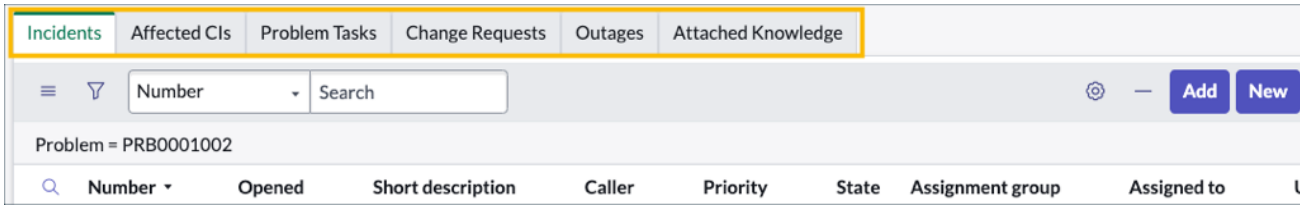
Làm thế nào ServiceNow biết phải làm gì? Quyết định được đưa ra bởi tùy chọn Reverse if false trong trigger UI Policy.

- Nếu Reverse if false được chọn (mặc định), hành động ngược lại được thực hiện trong UI Policy Actions. Nếu trường là bắt buộc (true), trường sẽ không còn bắt buộc (false). Nghĩa là, thuộc tính true thành false, và false thành true. Không có thay đổi cho thuộc tính được đặt Leave alone.
- Nếu Reverse if false không được chọn, không có logic UI Policy Action nào được áp dụng.

Tạo UI Policy Related List Actions

Sử dụng UI Policy Related List Actions để hiển thị hoặc ẩn danh sách liên quan. Form Problem có nhiều danh sách liên quan:

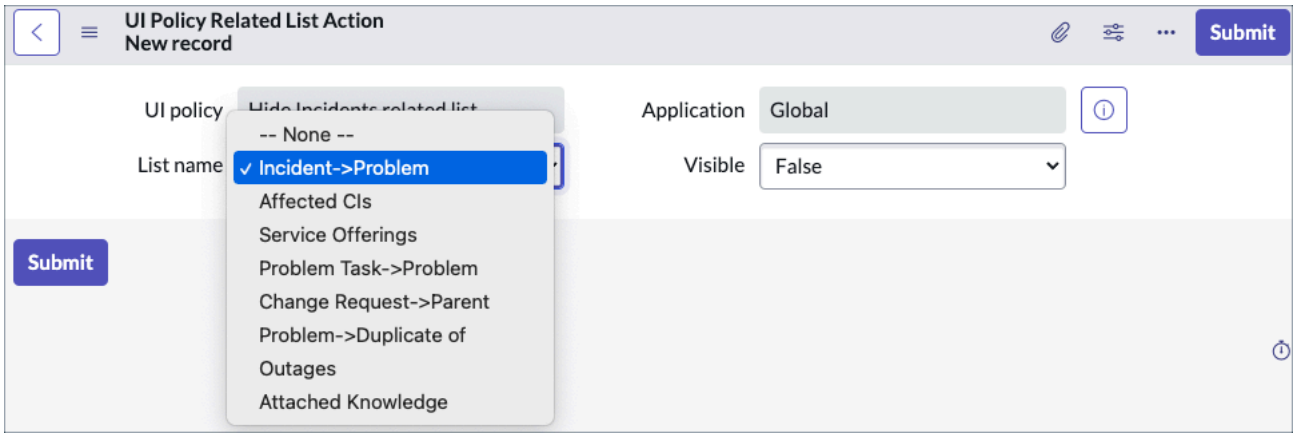
- Incidents
- Affected CIs
- Problem Tasks
- Change Requests
- Outages
- Attached Knowledge



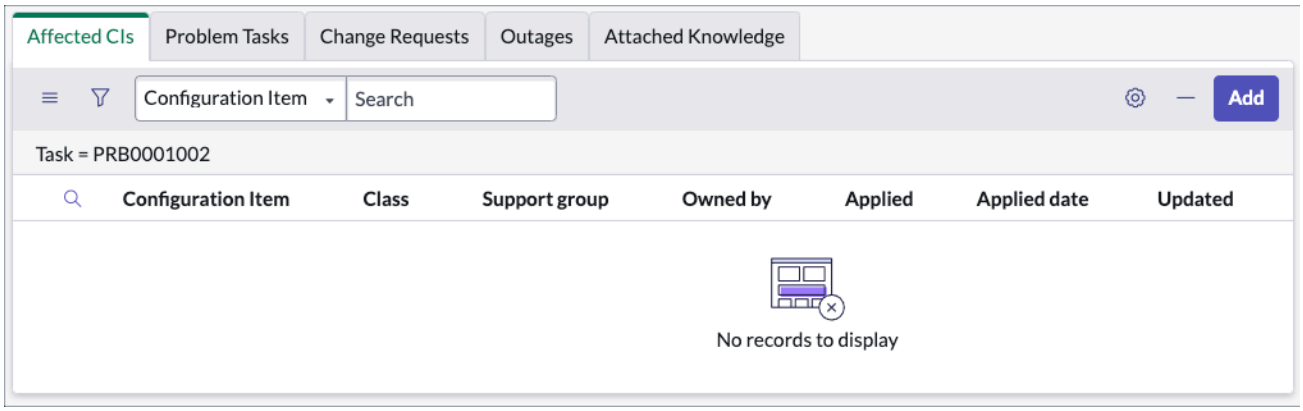
1. Trong Studio, tạo UI Policy hoặc mở UI Policy hiện có để chỉnh sửa.
2. Cuộn xuống danh sách liên quan UI Policy Related List Actions. Nếu tạo UI Policy mới, UI Policy phải được lưu trước khi danh sách liên quan UI Policy Related List Actions hiển thị.
3. Nhấp nút New.
4. Cấu hình UI Policy Related List Action.

1. Chọn List name.
2. Thiết lập giá trị trường Visible.
3. True: Áp dụng thuộc tính cho danh sách liên quan.
   False: Không áp dụng thuộc tính cho danh sách liên quan.
   Leave alone: Thuộc tính không áp dụng cho danh sách liên quan.

5. Nhấp nút Submit.



Danh sách liên quan Incidents bị ẩn:



BÀI VIẾT (15 TRÊN 21)

UI Policy Scripts

UI Policy scripts sử dụng API phía client để thực thi logic script dựa trên điều kiện UI Policy kiểm tra true hay false. Sử dụng UI Policy scripts để tạo kiểm tra điều kiện phức tạp hoặc thực hiện hành động khác ngoài thiết lập thuộc tính trường (bắt buộc, chỉ đọc, hoặc hiển thị).

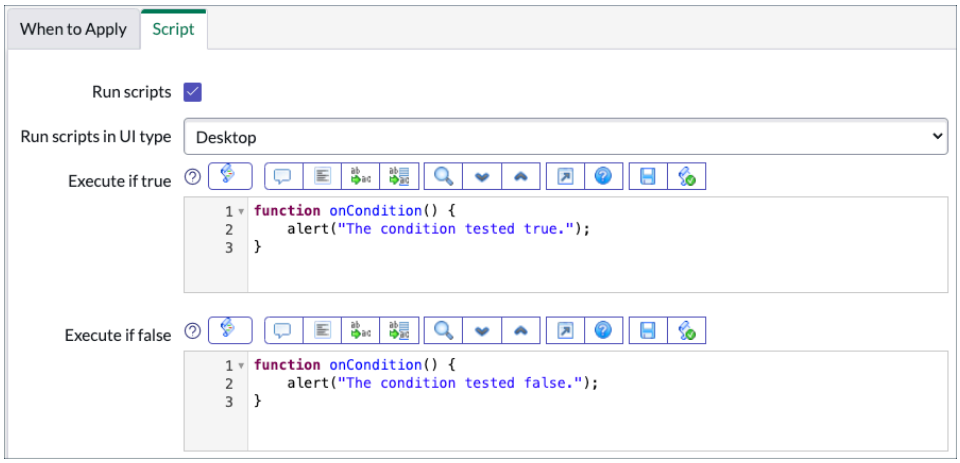
Các trường scripting cho UI Policies chỉ hiển thị trong view Advanced. Để bật các trường scripting, chọn tùy chọn Run scripts.

- Script Execute if true thực thi khi điều kiện UI Policy kiểm tra true.

- Script Execute if false thực thi khi điều kiện UI Policy kiểm tra false.

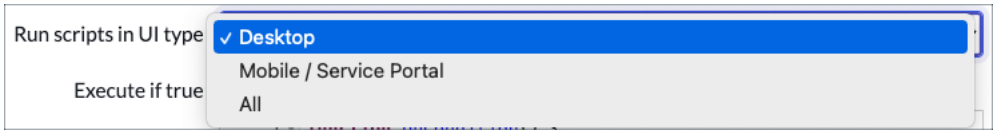
Developers có thể viết scripts trong một hoặc cả hai trường script tùy theo yêu cầu ứng dụng.

Viết logic script bên trong hàm onCondition được tự động chèn vào các trường scripting. Hàm onCondition() được gọi bởi UI Policy khi chạy.



QUAN TRỌNG: Tùy chọn Reverse if false phải được chọn để script Execute if false chạy.

Mặc dù UI Policy Actions thực thi trên tất cả nền tảng, UI Policy Scripts chỉ thực thi trên Desktop/tablet trong trường hợp mặc định. Dùng trường Run scripts in UI type để chọn nền tảng cho UI Policy scripts.



BÀI TẬP (16 TRÊN 21)

Bài tập: Tạo UI Policies

Trong bài tập này, bạn sẽ viết, kiểm thử và gỡ lỗi UI Policy.

- Tạo UI Policy Action để làm trường Other hiển thị và bắt buộc
- Viết script hướng dẫn người dùng nhập gì vào trường Other

Chuẩn Bị

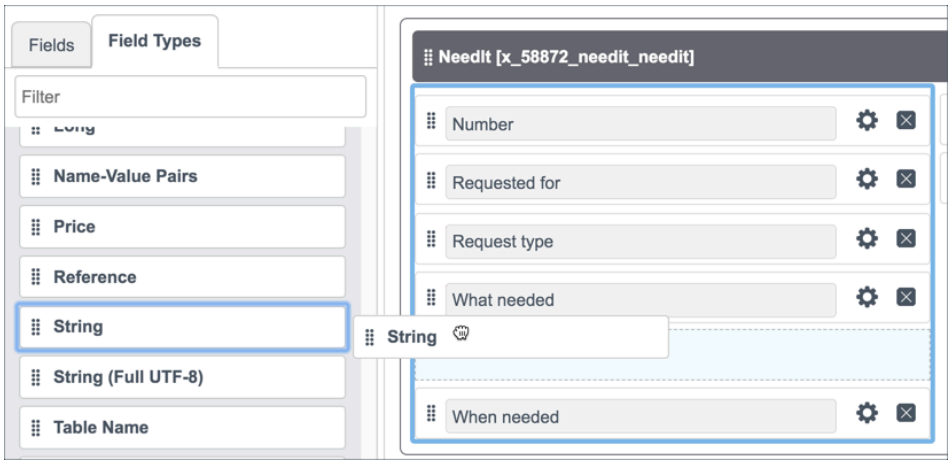
Danh sách lựa chọn What needed có giá trị Other. Khi Other được chọn, người dùng nên cung cấp thêm thông tin về những gì họ cần bằng cách giải thích yêu cầu trong trường mới gọi là Other.

1. Thêm trường vào bảng NeedIt.

1. Trong Studio, dùng Application Explorer để mở Forms & UI > Forms > NeedIt [Default view].

2. Trong Field Navigator, chọn tab Field Types.

3. Kéo loại trường String và thả vào form NeedIt giữa trường What needed và When needed.



4. Di chuột qua trường New String và nhấp nút Edit Properties (⚙).

5. Cấu hình Field Properties cho trường mới:

   1. Label: Other
      Name: u_other

6. Đóng hộp thoại Properties bằng cách nhấp nút Close (✕).

2. Nhấp nút Save.

Tạo UI Policy

1. Tạo UI Policy.

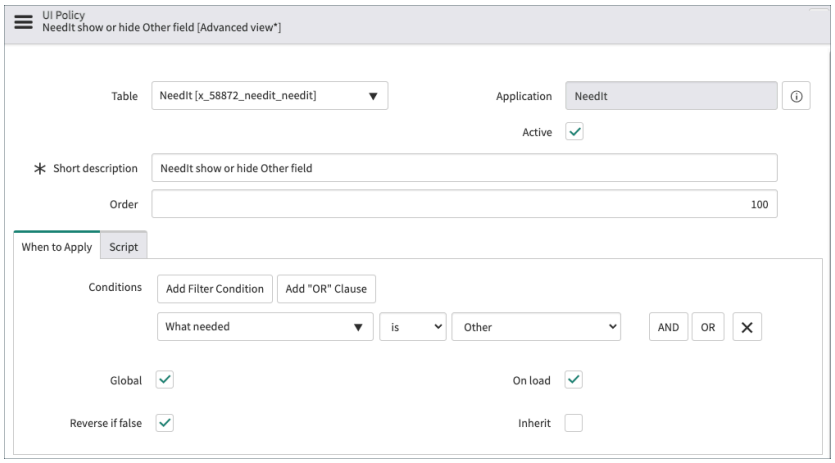
   1. Trong Studio, nhấp liên kết Create Application File.
   2. Trong trường Filter... nhập text UI Policy HOẶC chọn Client Development từ danh mục trong khung bên trái.
   3. Chọn UI Policy ở khung giữa làm loại file, sau đó nhấp nút Create.

2. Cấu hình UI Policy:

    1. Table: NeedIt [x_<mã_công_ty_của_bạn>_needit_needit]
       Active: Đã chọn (checked)
       Short Description: NeedIt hiển thị hoặc ẩn trường Other

3. Chuyển sang phần When to Apply và tiếp tục cấu hình UI Policy:

    1. Condition: [What needed] [is] [Other]
       Global: Đã chọn (checked)
       Reverse if false: Đã chọn (checked)
       On load: Đã chọn (checked)
       Inherit: Không chọn (not checked)

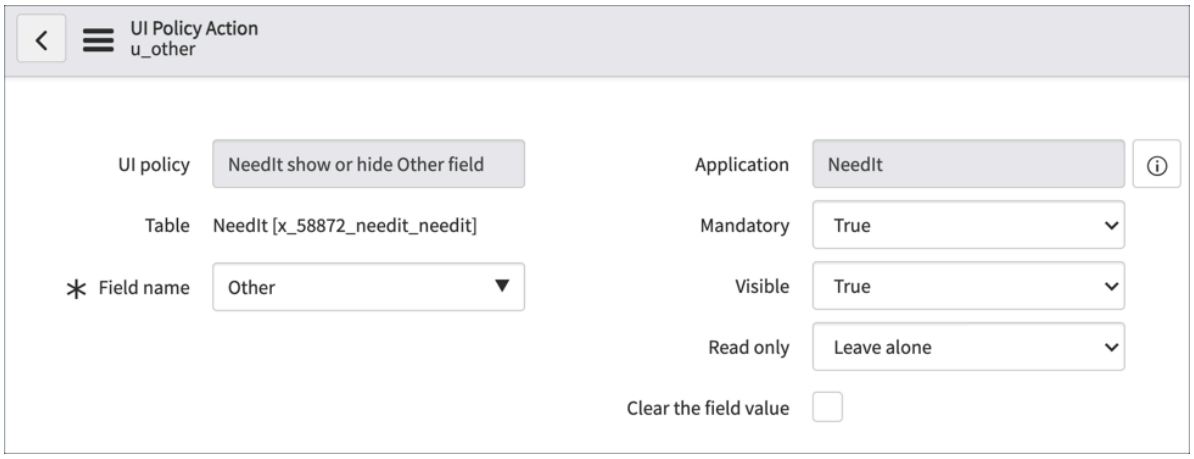


4. Nhấp nút Submit.

Tạo UI Policy Action

1. Trong UI Policy NeedIt show or hide Other field, cuộn xuống phần UI Policy Action.
2. Nhấp nút New.
3. Cấu hình UI Policy Action.
    1. Field name: Other
       Mandatory: True

Visible: True



4. Nhấp nút Submit.

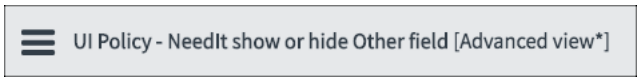
Kiểm thử UI Policy và UI Policy Action

1. Trong cửa sổ trình duyệt ServiceNow chính (không phải Studio), dùng nút reload của trình duyệt để tải logic phía client mới nhất vào trình duyệt.
2. Dùng menu All để mở NeedIt > Create New.
3. Đặt giá trị trường What needed thành Other.
4. Trường Other có xuất hiện không? Có bắt buộc không? Nếu không, gỡ lỗi và kiểm thử lại.

5. Thay đổi giá trị trường What needed thành bất cứ gì trừ Other.
6. Trường Other có biến mất không? Nếu không, gỡ lỗi và kiểm thử lại.

Tạo Script Execute if True của UI Policy

1. Quay lại tab Studio.
2. Nếu chưa mở, mở UI Policy NeedIt show or hide Other field để chỉnh sửa.
3. Nhìn header form để xem view có phải Advanced hay không. Nếu không, nhấp liên kết Related Advanced view.



4. Chuyển sang phần Script (tab).
5. Chọn tùy chọn Run scripts (checked).

6. Đặt giá trị Run scripts in UI type thành All.

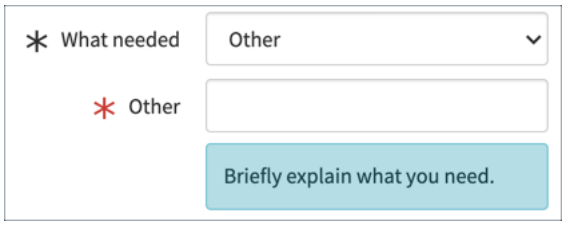
7. Thay thế nội dung trường script Execute if true bằng script này:

```
function onCondition() {
                    // Hiển thị thông báo dưới trường Other để giải thích nhập gì vào
                    // trường Other.
                    g_form.showFieldMsg('u_other','Briefly explain what you need.','info');
          }
```

8. Nhấp nút Update.

Kiểm thử Script Execute if True

1. Trong cửa sổ trình duyệt ServiceNow chính (không phải Studio), dùng menu All để mở NeedIt > Create New.
2. Đặt giá trị trường What needed thành Other.
3. Thông báo có xuất hiện dưới trường Other không? Nếu không, gỡ lỗi và kiểm thử lại.



4. Thay đổi giá trị trường What needed thành bất cứ gì trừ Other.
5. Thay đổi giá trị trường What needed trở lại Other.

Thử thách: Tạo Script Execute if False của UI Policy

Viết script trong trường Execute if false để xóa thông báo viết bởi script Execute if true. Sử dụng phương thức GlideForm hideFieldMsg(). Mô tả phương thức hideFieldMsg() có sẵn trong tài liệu API: GlideForm hideFieldMsg() documentation (https://developer.servicenow.com/app.do#!/api_doc?v=utah&id=r_GlideFormHideFieldMsg_String_Boolean). Thêm comment trong script để giải thích script làm gì.

Kiểm thử script để đảm bảo thông báo Briefly explain what you need. đã được xóa.

BÀI VIẾT (17 TRÊN 21)

Client Scripts vs. UI Policies

Client Scripts và UI Policies đều thực thi logic phía client và sử dụng cùng API. Cả hai đều dùng để quản lý forms và các trường. Khi phát triển ứng dụng, làm thế nào để quyết định loại script phía client nào sử dụng? Dùng bảng này để xác định loại nào phù hợp nhất với nhu cầu ứng dụng:

Tiêu chí	Client Script	UI Policy
Thực thi khi form tải	Có	Có
Thực thi khi lưu/gửi/cập nhật form	Có	Không
Thực thi khi giá trị trường thay đổi	Có	Có
Truy cập giá trị cũ của trường	Có	Không
Thực thi sau Client Scripts	Không	Có
Thiết lập thuộc tính trường không cần script	Không	Có
Yêu cầu kiểm soát thứ tự thực thi *	Có	Có

*Mặc dù trường Order không có trên Client Script form baseline, bạn có thể tùy chỉnh form để thêm.

UI Policies thực thi sau Client Scripts. Nếu có logic xung đột giữa Client Script và UI Policy, logic UI Policy được áp dụng.

BÀI TẬP (18 TRÊN 21)

Bài tập: Lưu Công Việc Lập Trình Phía Client (Tùy chọn)

Ứng dụng source control, như GitHub, cho phép developers commit thay đổi (lưu công việc hoàn thành) ngoài Personal Developer Instance (PDI). Commit thay đổi cho ứng dụng để lưu công việc trong source control.

Trong bài tập này, bạn sẽ lưu công việc hoàn thành trong module này vào repository GitHub.

> GHI CHÚ: Xem Hướng dẫn GitHub (/dev.do#!/guide/utah/now-platform/github-guide/github-and-the-developer-site-training-guide-introduction) để biết thêm thông tin về cách ServiceNow sử dụng GitHub với nội dung học tập Developer Program và xem video về cách để lưu công việc.

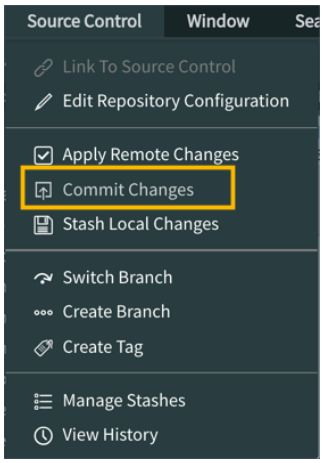
Commit Thay Đổi

1. Nếu ứng dụng NeedIt chưa mở trong Studio, hãy mở ngay.

    1. Trong cửa sổ trình duyệt ServiceNow chính, sử dụng menu All để mở System Applications > Studio.

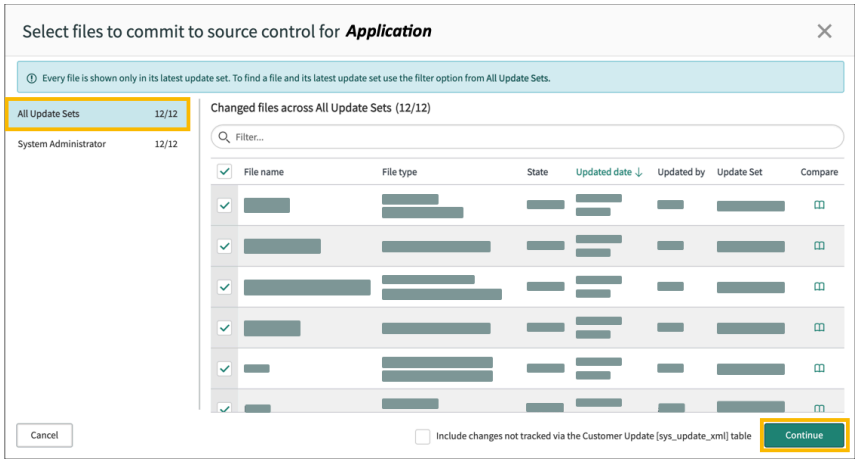
2. Trong hộp thoại Select Application, nhấp ứng dụng.

2. Mở menu Source Control và chọn mục Commit Changes.



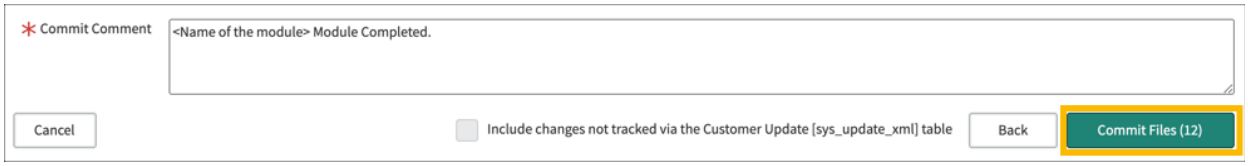
3. Chọn các cập nhật để commit.

   1. Trong hộp thoại Select files to commit to source control cho <Application>, chọn All Update Sets.
   2. Xem lại các file ứng dụng sẽ được commit.
   3. Nhấp nút Continue.



4. Trong hộp thoại Confirm files to commit to source control cho NeedIt, nhập Commit comment, ví dụ Client-side Scripting Module Completed.

5. Nhấp nút Commit Files.



6. Khi hộp thoại Commit Changes báo thành công, nhấp nút Close.

GHI CHÚ: Nếu commit thay đổi thất bại, có thể bạn đã nhập URL repository ServiceNow thay vì URL repository đã fork trong trường URL. Xem Troubleshooting GitHub Issues (/dev.do#!/guide/utah/now-platform/github-guide/troubleshooting-github-issues) trong Hướng dẫn GitHub để biết hướng dẫn khắc phục sự cố kết nối GitHub.

BÀI VIẾT (19 TRÊN 21)

Kiểm Tra Kiến Thức Lập Trình Phía Client

Muốn xác minh sự hiểu biết của bạn về lập trình phía client? Những câu hỏi này sẽ giúp bạn đánh giá tiến trình. Với mỗi câu hỏi, xác định câu trả lời rồi nhấp vào bất kỳ đâu trong câu hỏi để xem đáp án.

BÀI VIẾT (20 TRÊN 21)

Tóm Tắt Module Lập Trình Phía Client

Khái Niệm Cốt Lõi:

- Client-side scripts thực thi logic script trong trình duyệt web
- Client-side scripts quản lý forms và trường form
- Client Scripts thực thi logic script khi forms được:
  - Tải (Loaded)
  - Thay đổi (Changed)
  - Gửi/Lưu/Cập nhật
- UI Policies có điều kiện là một phần của trigger
- UI Policies có thể thực hiện hành động khác nhau khi điều kiện trả về true hoặc false
- UI Policy Actions không yêu cầu viết script để thiết lập thuộc tính trường:
  - Bắt buộc (Mandatory)
  - Hiển thị (Visible)
  - Chỉ đọc (Read only)
- UI Policy Related List Actions không yêu cầu viết script để hiển thị hoặc ẩn danh sách liên quan
- GlideForm API cung cấp phương thức để tương tác với forms và trường form
- GlideUser API cung cấp phương thức và thuộc tính để truy cập thông tin về người dùng đang đăng nhập và vai trò của họ

BÀI VIẾT (21 TRÊN 21)

Sau Khi Hoàn Thành Lập Trình Phía Client, Bạn Có Thể Quan Tâm Đến...

Chúc mừng bạn đã hoàn thành module Lập Trình Phía Client. Dựa trên sở thích về scripting, bạn có thể cũng thích:

- Server-Side Scripting (https://developer.servicenow.com/to.do?u=CSS-U-MOD-SSS): Trong module học tập Developer Site này, bạn sẽ học viết, kiểm thử và gỡ lỗi server-side scripts sử dụng Now Platform server-side scoped API.
- Data Policies (https://developer.servicenow.com/to.do?u=CSS-U-MOD-DAP): Trong module học tập Developer Site này, bạn sẽ học viết, kiểm thử và gỡ lỗi Data Policies.
- Client-side Scripting Technical Best Practices (https://developer.servicenow.com/to.do?u=CSS-U-TBP-ClientSideTBP): Trong Hướng dẫn Technical Best Practices của Developer Site này, bạn sẽ học các thực hành tốt nhất cho lập trình phía client trên Now Platform.
- Client-side Scripting Documentation (https://developer.servicenow.com/to.do?u=CSS-U-DOC-ClientSideScripting): Trên trang tài liệu ServiceNow, bạn sẽ tìm thấy bộ tài liệu tham khảo đầy đủ cho lập trình phía client.